M-A · Runtime, before any LLVM

6 tasks · A1 to A6

You are building the algorithmic core of CapsLock before touching a compiler. Everything in this milestone is ordinary C. If you cannot make the algorithm work standalone, you will not be able to debug it through an instrumentation pass. Do not skip ahead to LLVM because it feels like the real work.

Gate A

Runtime validated with no LLVM involved. You can explain revoke-on-use at a whiteboard without notes.

HOW TO USE THIS FILE

Fill in the ANSWER box under each task and return this `.tex` file. Leave everything else untouched so the briefs stay comparable. Tick one status per task. Where the question asks for numbers, output or commit hashes, paste them into the `verbatim` block rather than describing them in prose. If a task is blocked, say what by; a blocked task and a slow task need different responses.

Compile with `pdflatex`. Return the source, not the PDF.

Name	Zephyr Cheng
Date submitted	13 August 2026
Toolchain commit	`https://github.com/X-Bulow/capslock-artefacts`
Branch / worktree	`m-a-runtime`

A1 Reproduce the artefact

OBJECTIVE

Build capslock-qemu and the instrumented toolchain from the published artefact, and confirm it detects known violations.

WHY THIS MATTERS

This build is your ground truth for the entire project. Every claim you make later is measured against what it does. If you cannot reproduce it you cannot tell your own bugs from expected behaviour.

METHOD

1. Pull or build the Docker image (./pull-docker or ./build-docker). Allow roughly 10 GB of disk.
2. Run ./docker-test. The helloworld program must print its greeting; the violation program must report an attempt to use an invalid capability for load.
3. Run ./docker-eval2 and compare the output against Table 4 of the paper.
4. Record the exact commit hashes of capslock-qemu, capslock-rust and capslock-llvm that you built.

WATCH OUT FOR

— Your user id must be at least 1000 for the Docker scripts to work.

— Do not skip recording the commit hashes. You will need them for every differential comparison from M-G onward.

DONE WHEN

docker-eval2 detects all 15 RustSec PoCs.

YOU WILL BE ASKED

Did ./docker-eval2 detect all 15 RustSec PoCs, and how long did a clean build take end to end?

Status: `[ ]` not started `[ ]` in progress `[X]` done `[ ]` blocked

ANSWER — A1

Yes. The combined CapsLock result detected all 15 RustSec PoCs. The published script hard-codes 16 parallel QEMU jobs, which exceeded the 8 GB Docker VM under Apple Silicon amd64 emulation and killed eight jobs; each of those eight detected correctly when rerun sequentially. A successful no-cache build took 5776.11 seconds (1:36:16.11) after limiting the LLVM and Rust builds to two jobs. The timing is specific to the emulated environment.

Evidence — paste numbers, output or hashes below.

```
CapsLock: 15/15 RustSec PoCs detected
Clean build: real 5776.11 s

qemu upstream base + patch:
  6bb4a8a47a43f35a345f107227fcd6abed59e62c
llvm upstream base + patch:
  5399a24c66cb6164cf32280e7d300488c90d5765
rust upstream base + patch:
  c69fda7dc664e62f8920a02a4e55d6207b212c24

The artefact applies setup/patches/{qemu,llvm,rust}.patch to
these upstream revisions; it does not contain separate fork commits.
```

A2 Locate the LLVM submodule rewire

OBJECTIVE

Work out how capslock-llvm is substituted for upstream LLVM during the build.

WHY THIS MATTERS

The .gitmodules file in capslock-rust still declares src/llvm-project as rust-lang/llvm-project, branch rustc/18.0-2024-02-13. The CapsLock LLVM fork is not referenced there at all. Something in the setup path rewires it, and you need to know what before you can build a modified toolchain.

METHOD

1. Grep the artefact setup directory for llvm-project, submodule and capslock-llvm.
2. Check config.toml or bootstrap.toml for an llvm-config path or an external LLVM setting.

3. Record the file, the line, and the capslock-llvm commit it lands on.

WATCH OUT FOR

— If nothing rewires it, the build may be using a prebuilt LLVM. Find out which, and where it came from.

DONE WHEN

You can name the file and line performing the substitution.

YOU WILL BE ASKED

Which file or script repoints src/llvm-project to capslock-llvm, and which commit does it land on?

Status: `[ ]` not started `[ ]` in progress `[X]` done `[ ]` blocked

ANSWER — A2

Nothing repoints Rust's `src/llvm-project` submodule.
The artefact bypasses it. `setup/setup.py` fetches the pinned upstream LLVM tree and applies `setup/patches/llvm.patch`; `setup/build/llvm.sh` then installs that patched tree separately. During the Rust build, `rust.sh` generates `config.toml` with target-specific `llvm-config` entries pointing at the separate installation. Thus the substitution is through Rust's external LLVM configuration, not `.gitmodules`.

Evidence — paste numbers, output or hashes below.

```
setup/setup.py:9
  llvm upstream revision:
  5399a24c66cb6164cf32280e7d300488c90d5765
setup/setup.py:22
  patch -p1 < ../patches/llvm.patch
setup/build/rust-config.toml:11
  download-ci-llvm = false
setup/build/rust-config.toml:31,34
  llvm-config = "LLVM_LOCATION/installation/bin/llvm-config"
setup/build/rust.sh:2
  replaces LLVM_LOCATION and writes rust/config.toml
resolved path in Docker:
  /capslock-tools/llvm/installation/bin/llvm-config
```

A3 Decompose the overhead

OBJECTIVE

Measure where the time actually goes in the existing implementation.

WHY THIS MATTERS

Three multiplicative costs are conflated in the published performance numbers: cross-ISA emulation, unoptimised guest code, and the capability checking itself. Only the third is intrinsic to the idea. If you optimise before you measure, you may spend months on the smallest term.

METHOD

1. Pick one workload and keep it fixed for the whole project. Decompression in zlib-rs is a reasonable choice because it is pointer-chasing dense.
2. Time it four ways: (a) native x86-64; (b) stock qemu-riscv64 running the unoptimised RISC-V binary; (c) capslock-qemu with the capability instructions stubbed to no-ops; (d) full capslock-qemu.
3. Produce a ratio table. Write down the exact command lines and the host machine.

WATCH OUT FOR

— Use the same input and iteration count for all four measurements.

— You will be asked to reproduce this measurement identically at task G1. Document the method now, not later.

DONE WHEN

Four wall-clock numbers and a ratio table.

YOU WILL BE ASKED

What are the four numbers, and what fraction of the slowdown is capability checking rather than emulation?

Status: `[ ]` not started `[ ]` in progress `[X]` done `[ ]` blocked

ANSWER — A3

The fixed workload performed 100 compression/decompression iterations per measurement on a separate x86-64 machine. Each configuration was measured five times; the values below are arithmetic means. Full checking is 22.133× the no-op CapsLock configuration. Under an absolute elapsed-time decomposition, checking contributes 95.526% of the full-over-native slowdown: $(T_{full} - T_{stub})/(T_{full} - T_{native})$. The complementary 4.474% includes no-op CapsLock, instrumented-guest and emulation costs, so it must not be labelled pure cross-ISA emulation overhead.

Evidence — paste numbers, output or hashes below.

```
variant                          mean seconds   ratio/native
native x86-64                       0.014996        1.000x
stock qemu-riscv64                  0.170234       11.352x
CapsLock QEMU, checks stubbed       1.456172       97.104x
full CapsLock QEMU                 32.230098     2149.246x

full / stub = 22.133x
(full - stub) / (full - native) = 95.526%
(stub - native) / (full - native) = 4.474%
```

A4 Runtime core

OBJECTIVE

Write libcapslock.c implementing revoke-on-use, with no compiler involvement whatsoever.

WHY THIS MATTERS

This is the algorithmic heart of the project. It is ordinary C and can be fully unit-tested standalone. Debugging it later through an instrumentation pass is considerably harder.

METHOD

1. Structures: an allocation table mapping address range to tree root; a node arena; per-node parent, child and sibling links; bounds; and a permission of RW, RO or NA.
2. Implement borrow(node, mutable, base, end) returning a new node.
3. Implement access(node, range, is_write) returning whether the access is allowed, following the Store and Load rules in Table 2 of the paper: invalidate the subtrees of capabilities that overlap the accessed address and are not ancestors of the accessing capability.
4. Implement revoke(node), invalidating the subtree.
5. Use a hash map for shadow memory. Do not optimise anything yet.
6. Read, but do not copy: target/riscv/cap_rev_tree.c and .h in capslock-qemu, and borrow_impl() at target/riscv/op_helper.c line 709.

WATCH OUT FOR

— Walk from the accessing node up to the root, killing sibling subtrees as you go. Do not scan the whole tree. That difference is amortised constant time per access versus linear.

— Appendix A of the paper defines RevPermW and RevPermR with the two cases inverted relative to the prose in section 4.2. The prose is correct: capabilities in the set become NA.

DONE WHEN

C unit tests pass for borrow, access, revoke and nested borrow.

YOU WILL BE ASKED

Driven only by direct C calls, does your runtime accept a nested borrow chain and reject a use-after-revoke? Paste the test output.

Status: `[ ]` not started `[ ]` in progress `[X]` done `[ ]` blocked

ANSWER — A4

Yes. Driven only by direct C calls, the standalone runtime accepts a valid nested borrow chain and rejects a later access through a revoked descendant. It implements the allocation table, node arena, parent/child/sibling borrow trees, bounds, RW/RO/NA permissions, shadow memory, borrow, access and subtree revocation. The complete standalone A4/A5 suite passes 9/9 tests.

Evidence — paste numbers, output or hashes below.

```
[PASS] create and shadow map
[PASS] borrow permissions and bounds
nested borrow chain: accepted
use-after-revoke: rejected (attempting to use an invalid capability)
[PASS] nested borrow and revoke
[PASS] store invalidation
[PASS] load invalidation
[PASS] disjoint partial borrows
```

```
[PASS] RAW versus REF
[PASS] UnsafeCell relaxation
[PASS] access checks
9/9 runtime tests passed
```

A5 Node type bits

OBJECTIVE

Add the compatibility adjustments described in section 5.1 of the paper.

WHY THIS MATTERS

Without these, real Rust code produces false positives immediately. Raw pointers and interior mutability both need relaxed rules.

METHOD

1. Add a type field with three values mirroring the reference implementation: REF, RAW and UNSAFECELL. See cap_rev_node_type_t in cap_rev_tree.h.
2. Raw pointers: while a raw-pointer capability is valid, no store may alias it unless through a capability derived from its parent.
3. Interior mutability: except aliasing stores originating within the subtree of an UnsafeCell capability.

WATCH OUT FOR

— The reference tracks UnsafeCell subtrees in a separate hash table with previous and next links. You probably do not need that complexity yet, but read it before designing your own.

DONE WHEN

Tests cover raw-pointer relaxation and interior mutability.

YOU WILL BE ASKED

Which of the three node types are implemented, and what test distinguishes RAW from REF behaviour?

Status: `[ ]` not started `[ ]` in progress `[X]` done `[ ]` blocked

ANSWER — A5

All three node types are implemented: REF, RAW and UNSAFECELL. The RAW-versus- REF test creates RAW and REF siblings with identical bounds, then stores through a third capability derived from their common parent. The REF subtree becomes NA, whereas RAW remains RW because the store originated inside its parent's subtree. A separate test verifies that a store below an UNSAFECELL capability receives the interior-mutability relaxation.

Evidence — paste numbers, output or hashes below.

```
[PASS] RAW versus REF
[PASS] UnsafeCell relaxation
```

A6 Hand-written oracle programs

OBJECTIVE

Encode the paper's examples as plain C that calls your runtime by hand.

WHY THIS MATTERS

This is the step people skip, and skipping it makes M-B considerably harder. Once you have written the instrumentation by hand, the compiler pass's job becomes concrete: generate the code you already wrote.

METHOD

1. Encode the three cases in Listing 1: use-after-free, data race, and unsafe aliasing.
2. Encode Figure 3: create, then borrow to get v_raw, then borrow again to get v_ref, then store through v_raw, then use v_ref. The last use must be rejected.
3. Add five reduced RustSec patterns taken from the PoCs in the artefact.
4. For each case write a clean variant that must pass.

WATCH OUT FOR

— If a clean variant fails you have a false positive. Those matter more than misses. Fix them first.

DONE WHEN

Each case flags at the correct line; each clean variant passes.

YOU WILL BE ASKED

Which of the eight hand-written cases flag correctly? For each miss, is the cause the algorithm or the test encoding?

Status: `[ ]` not started `[ ]` in progress `[X]` done `[ ]` blocked

ANSWER — A6

The method actually requests nine cases (three from Listing 1, one from Figure 3, and five RustSec reductions), although the final question says eight. All nine flag at their annotated direct-C access line: Listing 1(a) use-after-free, Listing 1(b) data race, Listing 1(c) unsafe aliasing, Figure 3, RUSTSEC-2020-0023, RUSTSEC-2022-0002, RUSTSEC-2021-0114, RUSTSEC-2019-0009 and RUSTSEC-2022-0007. There are no misses to classify as an algorithm or encoding error, and all nine clean variants pass.

Evidence — paste numbers, output or hashes below.

```
9/9 buggy cases flagged; 9/9 clean variants passed
```

GATE A -- SIGN-OFF

Runtime validated with no LLVM involved. You can explain revoke-on-use at a whiteboard without notes.

`[X]` Gate met `[ ]` Not met

If not met, state which task is outstanding and why: